Normalization

Database normalization is the process of organizing data into related tables so that each fact is stored where it belongs. For example, a `Section` table records which course a section belongs to, while an `Enrollment` table records which students take that section. Storing the course assignment once avoids repeating it for every student and reduces the risk of inconsistent updates. **Normal forms**, such as 1NF, 2NF, and 3NF, provide rules for deciding how to structure those tables.

Functional Dependency

A **functional dependency** `X → Y` means that whenever two rows agree on the values of `X`, they must also agree on the values of `Y`. Either side can contain one or more attributes.

For example, in an `Enrollment` table with primary key `(StudentID, CourseID)`, a student can appear once per course.

StudentID	CourseID	Campus
101	CS101	North
101	MATH201	North

Candidate key and Superkey

A **candidate key** is a minimal set of attributes that uniquely identifies each row. It **determines every attribute** in the relation, and removing any attribute from it makes it lose that ability. A candidate key with more than one attribute is a **composite key**. If a relation has several candidate keys, one can be chosen as the primary key.

Consider **`Student(StudentID, UniversityEmail, FullName, Major)`**. Assume every student has a unique `StudentID` and a unique `UniversityEmail`, and both are required.

- **`StudentID` is a candidate key** because it uniquely identifies a student and determines the other attributes.
- **`UniversityEmail` is also a candidate key** for the same reason.

The pair `(StudentID, UniversityEmail)` also identifies a student, but it is a **superkey** because it contains more attributes than needed.

Trivial and Non-Trivial Functional Dependency

A functional dependency **`X → Y` is non-trivial** if at least one attribute on the right side is missing from the left side. It says that knowing `X` determines some additional information.

Dependency	Type	Why
`AB → A`	Trivial	If two rows agree on both `A` and `B`, they already agree on `A`.
`AB → C`	Non-trivial	`C` is outside `AB`, so the dependency adds a rule about `C`.
`AB → BC`	Non-trivial	`B` is already on the left, but `C` is an additional attribute.

Prime and non-prime attributes

- A **prime attribute** belongs to at least one candidate key.
- A **non-prime attribute** belongs to none of the candidate keys.

Example

Consider `R(A, B, C, D)` with `D → A`, `B → D`, and `C → A`.

- **`BC` is the only candidate key.** Every key must include `B` and `C` because neither can be derived from the given dependencies. Together they determine `D` through `B → D` and `A` through `C → A`.
- **`B` and `C` are prime attributes.** They belong to the candidate key.
- **`A` and `D` are non-prime attributes.** Neither belongs to a candidate key.

Consider the relation **`R(A, B, C, D)`**, where **`A` is the primary key**. The following functional dependencies hold:

- `A → B`
- `A → C`
- `A → D`

Each value of `A` uniquely identifies a row and determines its values of `B`, `C`, and `D`. You can write these dependencies together as

- **`R(A, B, C, D)`** with **`A → BCD`**

Or another way to state it is

- **`R(A, B, C, D)`** with **`A → B, A → C, A → D`**

For example, in `Student(StudentID, FirstName, LastName, Email)`, `StudentID` is the primary key. A student ID determines the first name, last name, and email recorded for that student.

Because `A` is a single attribute, these dependencies are on the whole primary key. None of them is a partial dependency.

Now, consider the relation `R(A, B, C, D)`, where **`(B, C)` is the composite primary key**. The two attributes together uniquely identify each row, so `BC → AD`.
Or another way to state it is: `BC → A, BC → D`

For example, in `Enrollment(Campus, CourseID, StudentID, RoomID)`, the composite primary key is **`(CourseID, StudentID)`**. Each course–student pair identifies one enrollment.

Assume each `CourseID` has one assigned room. Then `CourseID → RoomID` is a **partial dependency** because `CourseID` alone determines `RoomID` and is only part of the composite key.

Normal Forms in a Relational Database Design

Normal forms are rules for organizing tables in a relational database to reduce redundant data and prevent inconsistencies when rows are added, updated, or deleted.

The usual numbering goes from **First Normal Form (1NF) through Sixth Normal Form (6NF)**. **Boyce–Codd Normal Form (BCNF)** sits between 3NF and 4NF, so the commonly taught sequence has **seven** forms:

1NF → 2NF → 3NF → BCNF → 4NF → 5NF → 6NF

Most ordinary student and enrollment tables can stop at **3NF or better yet BCNF**.

First Normal Form (1NF)

A relation is in **First Normal Form (1NF)** when each attribute holds a single value from its domain in each row and no repeating records. In a student table, for instance, a `CourseID` column should hold one course ID per row. If a student takes several courses, record each enrollment in its own row.

Consider `R(A, B)`. If `A` is supposed to hold the student ID, and `B` is supposed to hold a course ID but one row stores a list such as `CS101, MATH120` in `B`, the relation violates 1NF.

StudentID	CourseIDs
1001	CS101, MATH120
1002	CS101

The first row stores two course IDs in one cell. To put this data in 1NF, use `Enrollment(StudentID, CourseID)` with one row per enrollment:

StudentID	CourseID
1001	CS101
1001	MATH120
1002	CS101

Both `StudentID` and `CourseID` can repeat individually. Together, **`(StudentID, CourseID)`** forms a composite primary key, assuming a student can enroll in a given course only once.

1NF concerns what each cell holds. **2NF** concerns whether a non-prime attribute depends on only part of a candidate key.

Another example, in `Enrollment(StudentID, CourseID)`, a student ID can appear in several rows because a student can take several courses. A course ID can also appear in several rows because many students can take the same course. If each student can enroll in a given course only once, the same **student–course pair** should appear only once. But if a student re-takes the same course (due to a re-take), then the two rows will be identical:

Student ID	Course
1001	Mathematics
1001	Mathematics
1002	Physics

To fix this, we need to add another column in our table to indicate that the same student took the same course on different semesters. So we add one more column.

Student ID	Course	Semester
1001	Mathematics	Spring 2026
1001	Mathematics	Fall 2026
1002	Physics	Spring 2026

Question: Consider `R(A, B, C, D)`. Suppose `B` represents a course ID, but one row stores `CS101, MATH120` in a single `B` cell. What is the **normal form** that this table fails to satisfy, and why?

Answer: The table violates **First Normal Form (1NF)** because the `B` cell holds two course IDs where the column is meant to hold one.

Question: Consider the SQL table `R(A, B, C, D)` with the following rows:

A	B	C	D
North	CS101	1001	R201
North	MATH120	1001	R305
North	CS101	1001	R201

What is the **normal form** that this table fails to satisfy, and why?

Answer: 1NF. The first and third rows match in all four columns, so they are duplicates.

Second Normal Form (2NF)

Second Normal Form (2NF) helps reduce duplicated information. A table is in 2NF when:

1. It is already in **First Normal Form (1NF)**.
2. Every **non-prime** attribute depends on the **whole of every candidate key**, rather than just part of one.

Example

Consider this enrollment table:

Student ID	Course ID	Student Name	Grade
101	C01	Alice Smith	A
101	C02	Alice Smith	B
102	C01	Bob Jones	B

The primary key is **Student ID + Course ID**, which uniquely identifies each enrollment.

- **Grade** depends on both the student and the course.
- **Student Name** depends only on Student ID.

Because Student Name depends on only part of the key, this is a **partial dependency**, and the table is **not in 2NF**. Alice's name is repeated for every course she takes, which can cause update anomalies.

To achieve 2NF, split the table:

Students

Student ID	Student Name
101	Alice Smith
102	Bob Jones

Enrollments

Student ID	Course ID	Grade
101	C01	A
101	C02	B
102	C01	B

Student names are now stored once, and each non-key attribute depends on the whole key of its table.

The above example can be condensed into:

R(A, B, C, D) and AB→D, A→C. This violates 2NF.

What is the highest form of this table?

Third Normal Form (3NF)

Third Normal Form (3NF) reduces duplication by removing **transitive dependencies**. In a typical table, this means non-prime attributes should depend on the key, rather than on another non-prime attribute.

A table is in 3NF when:

1. It is already in **Second Normal Form (2NF)**.
2. It has no transitive dependencies of non-prime attributes on a candidate key.

For example:

Student ID	Student Name	Department ID	Department Name
101	Alice Smith	D01	Computing
102	Bob Jones	D01	Computing
103	Carol King	D02	Business

The primary key is **Student ID**. However, Department Name depends on Department ID:

Student ID → Department ID → Department Name

This is a transitive dependency, so the table is **not in 3NF**. If Computing is renamed, every student row for that department must be updated, creating the possibility of an update anomaly.

A **transitive dependency** occurs when one attribute determines another **through an intermediate attribute**. If `A → B` and `B → C`, then `A → C` follows by transitivity.

To achieve 3NF, split the table:

Students

Student ID	Student Name	Department ID
101	Alice Smith	D01
102	Bob Jones	D01
103	Carol King	D02

Departments

Department ID	Department Name
D01	Computing
D02	Business

Each department's name is now stored once. Department ID links the two tables, so you can still find each student's department name.

Question: Consider the following relation `R(A, B, C)` with functional dependencies `A → B` and `B → C`. What is the first normal form that `B → C` violates?

Answer: `B → C` violates **Third Normal Form (3NF)**. The only candidate key is `A`, and `C` depends on `A` through `B` (`A → B → C`). This is a **transitive dependency**. Specifically, `B` is not a superkey, and `C` is a non-prime attribute.

The relation satisfies **2NF** because its candidate key `A` consists of a single attribute and there are no partial dependencies.

In the course example, `A = CourseID`, `B = RoomID`, and `C = Campus`. The dependency in question is `RoomID → Campus`.

Boyce–Codd Normal Form (BCNF)

A relation is in **Boyce–Codd Normal Form (BCNF)** if, for every **nontrivial** functional dependency `X → Y`, **x is a superkey**. In other words, whenever a set of attributes determines an attribute outside that set, it must also determine every attribute in the relation.

BCNF is stricter than **Third Normal Form (3NF)**. For a dependency `X → A` with a single attribute on the right, 3NF allows `A` to be a **prime attribute** when `X` is not a superkey. BCNF requires `X` to be a superkey even when `A` is prime.

Example

Consider `R(A, B, C)` with the functional dependencies `AB → C` and `C → B`.

- **`AB` and `AC` are candidate keys.** `AB` determines `C`, and `AC` determines `B` through `C → B`. Both are minimal.
- **`C → B` satisfies 3NF** because `B` belongs to the candidate key `AB`.
- **`C → B` violates BCNF** because `C` determines `B` but cannot determine `A`. Therefore, `C` is not a superkey.

So `R` is in **3NF**, but it fails **BCNF**.

Consider **`Enrollment(StudentID, CourseID, SectionID)`**. Assume each section belongs to exactly one course, and each student enrolls in at most one section per course.

Enrollment

StudentID	CourseID	SectionID
101	CSC1700	CSC1700-01
101	MATH2010	MATH2010-01
102	CSC1700	CSC1700-02

The functional dependencies are:

- **`StudentID, CourseID → SectionID`**. A student and a course determine which section the student is enrolled in.
- **`SectionID → CourseID`**. A section determines which course it belongs to.

The two **candidate keys** are `(StudentID, CourseID)` and `(StudentID, SectionID)`. The first pair determines the section, and the second pair determines the course. Each pair is minimal.

The dependency **`SectionID → CourseID`** shows the difference between the two normal forms:

Normal form	Why the table passes or fails
3NF	`CourseID` is a **prime attribute** because it belongs to the candidate key `(StudentID, CourseID)`. That satisfies the 3NF rule for this dependency.
BCNF	`SectionID` alone cannot determine `StudentID`, since many students can enroll in the same section. BCNF requires `SectionID` to be a superkey, so this dependency violates BCNF.

To put the table in **BCNF**, split `Enrollment(StudentID, CourseID, SectionID)` into two tables:

Enrollment

StudentID	SectionID
101	CSC1700-01
101	MATH2010-01
102	CSC1700-02

Section

SectionID	CourseID
CSC1700-01	CSC1700
CSC1700-02	CSC1700
MATH2010-01	MATH2010